

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики – процессов управления

Шаго Павел Евгеньевич

Выпускная квалификационная работа

**«Математическое моделирование и реализация
планировщика процессов для гетерогенных
процессорных архитектур»**

Уровень образования: бакалавриат

Направление: 01.03.02 Прикладная математика и информатика

Основная образовательная программа СВ.5005.2022: «Прикладная математика,
фундаментальная информатика и программирование»

Научный руководитель
кандидат физ.-мат. наук, доцент
Корхов Владимир Владиславович

Рецензент
Эксперт по архитектуре и сетевому стеку
ООО НИЦ АЭРОДИСК
Анохин Юрий Владимирович

Санкт-Петербург
2026

Содержание

Введение	3
Постановка задачи	4
1 Подходы к реализации планировщиков в ОС	5
1.1 Задача планирования	5
1.2 Гетерогенные процессорные архитектуры	6
1.3 Расширяемый планировщик Linux	7
1.4 Актуальные исследования планировщиков	10
1.5 Анализ подходов к реализации планировщика	13
2 Математические модели планировщиков	15
2.1 Введение	15
2.2 Обозначения и допущения	15
2.3 Earliest Eligible Virtual Deadline First sloppy	15
2.4 Аукционная модель A1349 sloppy	17
3 Реализация алгоритмов планирования	21
4 Описание тестового окружения и экспериментального оборудования	22
5 Анализ результатов эксперимента	23
Заключение	24
Список литературы	25

Введение

Постановка задачи

Объектом исследования являются алгоритмы планирования задач (процессов) в операционных системах на вычислительных устройствах с гетерогенной процессорной архитектурой (Intel Hybrid, ARM big.LITTLE и другие).

Целью работы является разработка и экспериментальная оценка планировщика, обеспечивающего более высокую эффективность вычислений на гетерогенном оборудовании по сравнению с базовым алгоритмом EEVDF ядра Linux.

Для достижения цели работы необходимо:

1. Проанализировать и определить наиболее перспективный подход к реализации планировщика.
2. Построить математическую модель, учитывающую различную вычислительную мощность ядер процессора.
3. Реализовать предложенный алгоритм с использованием инфраструктуры `sched_ext` и разработать воспроизводимый набор бенчмарков.
4. Провести эксперимент на гетерогенном оборудовании и оценить результаты в сравнении с базовым EEVDF.

1 Подходы к реализации планировщиков в ОС

1.1 Задача планирования

В современных операционных системах механизмы многозадачности являются фундаментальным инструментом управления вычислительными ресурсами. Реализация концепции псевдопараллелизма базируется на процедурах контекстного переключения, которые позволяют распределять кванты процессорного времени между потоками исполнения.

Ключевым вызовом в данном контексте является задача построения эффективного расписания, при котором порядок выполнения задач минимизирует простой оборудования и максимизирует целевые показатели эффективности. Обычно задачу построения такого расписания и дальнейшего распределения задач по ядрам процессора решает компонент непосредственно встроенный в ядро операционной системы называемый планировщиком.

С развитием операционных систем и оборудования планировщики также эволюционировали. Так, в операционной системе UNIX планировщик выбирал задачу по приоритету, а среди равных задач использовался алгоритм похожий на round-robin [1]. Позднее в ядре Linux версии 2.4 был реализован планировщик $O(n)$, который при планировании следующей задачи просматривал очередь готовых к исполнению задач и вычислял для них эвристику *goodness* [2]. В Linux 2.6 его сменил планировщик $O(1)$, в котором разделили очередь приоритетов на два массива, *active* и *expired*, что позволило выбирать следующую задачу за постоянное время [3].

Затем в версии Linux 2.6.23 произошёл переход к планировщику Completely Fair Scheduler (CFS), основанному на модели виртуального времени. Готовые к выполнению задачи хранились в красно-чёрном дереве, упорядоченном по виртуальному времени, а планировщик выбирал задачу, получившую меньше всего процессорного времени относительно своей справедливой доли [4]. В Linux 6.6 эта идея получила развитие в планировщике Earliest Eligible Virtual Deadline First (EEVDF), где вводятся отставание и виртуальный дедлайн. Допустимыми считаются задачи, у которых отставание $\text{lag} \geq 0$, среди них выбирается задача с ближайшим виртуальным дедлайном [5].

Таким образом, за последние полвека планировщик операционной системы прошёл путь от детерминированных приоритетных схем, близких по логике к round-robin, до сложных алгоритмов динамического распределения ресурсов с сильной математической базой.

1.2 Гетерогенные процессорные архитектуры

Параллельно с развитием операционных систем меняется и аппаратная платформа, на которой они исполняются. Если классические многопроцессорные системы строились на симметричном принципе (SMP), при котором все ядра обладают одинаковой производительностью и одинаковым энергопотреблением, то современные процессоры всё чаще объединяют на одном кристалле ядра разных классов. Архитектура ARM big.LITTLE впервые сделала такой подход массовым в мобильном сегменте, объединив производительные ядра (*big*) с энергоэффективными (*LITTLE*) [6]. В настольных и серверных процессорах Intel начиная с поколения Alder Lake внедрена гибридная архитектура с Performance и Efficient ядрами (P/E) [7]. Принцип не привязан к конкретной системе команд, аналогичные конфигурации прорабатываются и в экосистеме RISC-V, где появляются SoC, объединяющие на одном кристалле ядра разных классов производительности, например, гетерогенный SoC Marsellus [8].

Ключевая особенность подобных систем заключается в том, что ядра одного процессора отличаются по тактовой частоте, ширине внеочередного исполнения, размеру кэшей и удельному энергопотреблению. Одна и та же задача, выполненная на P-ядре, завершается быстрее, но обходится дороже по энергии, тогда как E-ядро обеспечивает лучшее соотношение производительность/ватт при более низкой пропускной способности. В подобных системах выбор ядра для исполнения задачи существенно влияет на итоговое поведение системы. Размещение чувствительной к задержкам задачи на E-ядре увеличивает время её обслуживания, а выполнение фоновой нагрузки на P-ядре приводит к избыточному энергопотреблению.

В ядре Linux различия между ядрами обрабатываются подсистемой Capacity Aware Scheduling (CAS), которая распознаёт неоднородные топологии и учитывает относительную мощность каждого ядра при балансировке. Поверх неё работает Energy Aware Scheduling (EAS), которая по модели энергопотребления и оценкам загрузки ядер выбирает для задачи ядро процессора с меньшим потреблением без потери пропускной способности. EAS включается лишь при выполнении ряда условий, среди которых наличие энергомодели, регулятор частоты schedutil и отсутствие SMT, поэтому применяется прежде всего на мобильных SoC семейства ARM big.LITTLE [9]. На стороне x86 для информирования планировщика о приоритете ядер задействуется механизм Intel Thread Director, транслирующий аппаратные подсказки о характере нагрузки в теги класса задачи [7]. Однако CAS и Thread Director закрывают лишь балансировку по вычислительной мощности и классификацию по профилю инструкций, тогда как отдельные очереди готовых задач для P/E-кластеров, приоритизация чувствительных к задержкам нагрузок и прикладные правила выбора

кластера остаются за пределами штатной подсистемы. Реализация подобных политик внутри fair-класса требует существенных изменений ядра, что мотивирует переход к расширяемым механизмам, рассматриваемым в следующем разделе.

1.3 Расширяемый планировщик Linux

`sched_ext` (scheduler extensions) [10] – инфраструктура (фреймворк) ядра Linux, позволяющая реализовывать политику планирования как eBPF-программу, подключаемую к интерфейсу расширяемых планировщиков во время работы системы и не требуя пересборки ядра. Фреймворк `sched_ext` был включён в основную ветку Linux в версии 6.12 и предоставляет отдельный класс планирования `ext_sched_class`, расположенный между классами `fair_sched_class` и `idle_sched_class`. При загруженном eBPF-планировщике обычные задачи переводятся из fair-класса под управление `sched_ext`.

Подход с вынесением политики планирования из ядра встречался и до `sched_ext`. В системе ghOSt [11] политика запускается как пользовательский процесс-агент, а ядро ОС исполняет полученные от агента решения о переключениях. Агент видит состояние задач через разделяемую память и отвечает за выбор задачи и процессорного ядра, тогда как ядро ОС берёт на себя лишь переключение контекста, что позволяет описывать политику на обычном языке программирования и опираться на пользовательские средства отладки.

Важно отметить, что `sched_ext` замещает только политику справедливого планирования (fair-класс). Задачи реального времени (`SCHED_FIFO`, `SCHED_RR`) и задачи с дедлайнами (`SCHED_DEADLINE`) остаются под управлением штатных классов `rt_sched_class` и `dl_sched_class` соответственно. Это ограничивает область применимости пользовательских политик, но одновременно сохраняет гарантии для критичных по времени задач.

При штатной выгрузке `sched_ext` планировщика, при наличии в нем критической ошибки или при срабатывании специального таймера ядро автоматически возвращает все задачи под управление EEVDF, что обеспечивает безопасность экспериментов с пользовательскими политиками непосредственно на работающей системе.

С точки зрения программной модели, политика планирования в `sched_ext` описывается набором обратных вызовов, объединённых в структуру `struct sched_ext_ops`. Основными обработчиками являются: `select_cpu`, вызываемый при пробуждении задачи и отвечающий за выбор ядра исполнения, `enqueue`, помещающий задачу в очередь готовых к выполнению, `dispatch`, выбирающий следующую задачу для конкретного ядра, а также `running`

и `stopping`, дающие дополнительные контрольные точки для обновления метрик (при пробуждении и снятии задачи). Дополнительные обработчики `init_task`, `exit_task`, `cpu_online` и `cpu_offline` позволяют поддерживать собственное состояние, связанное с задачами и ядрами, на протяжении всего их жизненного цикла. Отдельная группа обработчиков `cgroup_init`, `cgroup_exit`, `cgroup_move` и `cgroup_set_weight` обеспечивает интеграцию с иерархией контрольных групп: они вызываются при создании и удалении `cgroup`, перемещении задач между группами, а также при изменении веса группы, что позволяет реализовывать иерархические политики планирования.

Поведение фреймворка настраивается через флаги поля `ops->flags`. Флаг `SCX_OPS_SWITCH_PARTIAL` переводит под управление `sched_ext` только задачи с явной политикой `SCHED_EXT`, оставляя `SCHED_NORMAL`, `SCHED_BATCH` и `SCHED_IDLE` в `fair`-классе. Флаг `SCX_OPS_ENQ_LAST` определяет, попадает ли последняя выполнявшаяся задача обратно в очередь после исчерпания кванта. Флаг `SCX_OPS_KEEP_BUILTIN_IDLE` указывает ядру продолжать обновлять битовую маску простаивающих CPU при загруженной пользовательской политике. Без этого флага `eBPF`-программе пришлось бы вести учёт самостоятельно, тогда как с ним поиск свободного ядра доступен через штатные `kfunc`-вызовы.

Передача задач между обработчиками происходит через очереди диспетчеризации *dispatch queue* (DSQ). Помимо встроенных глобальной `SCX_DSQ_GLOBAL` и локальных по числу ядер `SCX_DSQ_LOCAL`, `eBPF`-программа может создавать произвольное количество собственных очередей с заданным идентификатором при помощи `scx_bpf_create_dsq`. Каждая DSQ обслуживается по принципу FIFO либо по виртуальному времени, задаваемому при вставке через `scx_bpf_dsq_insert_vtime`. В режиме виртуального времени очередь реализована как красно-чёрное дерево, упорядоченное по полю `vtime`, что обеспечивает извлечение задачи с наименьшим виртуальным временем за $O(\log n)$. Возможность создавать произвольные DSQ позволяет завести отдельную очередь на каждый кластер ядер и обслуживать её по собственному виртуальному времени. Тогда задача, помещённая в очередь нужного кластера на этапе `enqueue`, попадёт на исполнение только к ядрам этого кластера, а упорядочивание по `vtime` сохраняется внутри кластера независимо от остальных. Такое разделение существенно в контексте гетерогенных архитектур, где P/E-ядра отличаются по производительности и требуют отдельного учёта.

Для взаимодействия с ядром `eBPF`-программа использует набор `kfunc`-вызовов, образующих стабильный интерфейс планировщика: `scx_bpf_dsq_insert` помещает задачу в выбранную очередь, `scx_bpf_dsq_move_to_local` извлекает задачу из очереди для исполнения на текущем ядре. Состояние задач и ядер хранится в `BPF_MAP`-структурах,

а доступ к полям `task_struct` и `rq` осуществляется через механизм CO-RE (Compile Once – Run Everywhere), что обеспечивает переносимость скомпилированной программы между версиями ядра.

Для типичных нужд балансировки фреймворк предоставляет встроенный учёт простаивающих ядер: функции `scx_bpf_pick_idle_cpu` и `scx_bpf_test_and_clear_cpu_idle` позволяют без собственной реализации находить свободные CPU по битовой маске. Если обработчик `select_cpu` помещает задачу непосредственно в локальную очередь `SCX_DSQ_LOCAL` целевого ядра, последующий вызов `enqueue` для этой задачи пропускается, что образует быстрый путь пробуждения и сокращает накладные расходы на часто пробуждающихся задачах.

Собственное состояние, привязанное к конкретной задаче, удобно хранить в структуре типа `BPF_MAP_TYPE_TASK_STORAGE`, которая обеспечивает константное время доступа без хеш-поиска и автоматически освобождается при завершении задачи. Длительность кванта процессорного времени задаётся присваиванием поля `p->scx.slice` в обработчиках `enqueue` или `dispatch`, а значением по умолчанию служит `SCX_SLICE_DFL` (20 мс). Вытеснение уже исполняющейся задачи на удалённом ядре инициируется вызовом `scx_bpf_kick_cpu` с флагом `SCX_KICK_PREEMPT`.

Безопасность исполнения произвольной политики обеспечивается несколькими механизмами времени выполнения. Таймер `watchdog` отслеживает случаи, когда задача не получает процессорного времени дольше установленного порога (по умолчанию 30 секунд), и при срабатывании выгружает eBPF-планировщик. Дополнительно реализована возможность экстренного отключения пользовательской политики комбинацией клавиш `SysRq-S`.

На этапе загрузки программы среда eBPF накладывает на политику планирования ряд архитектурных ограничений, проверяемых верификатором. В программе запрещены блокирующие вызовы и динамическая аллокация памяти, любой цикл должен иметь верифицируемую верхнюю границу, а доступ к структурам ядра разрешён только через утверждённый набор `kfunc` и полей. Эти требования заметно сказываются на выборе структур данных и способов агрегирования метрик при разработке планировщика.

1.4 Актуальные исследования планировщиков

С появлением фреймворка `sched_ext` [10] исследовательская активность в области процессных планировщиков заметно возросла. Большинство современных работ используют `sched_ext` как экспериментальную платформу для быстрой проверки новых алгоритмов без модификации ядра. Диапазон применения охватывает оптимизации существующих алгоритмов, подходы машинного обучения и нестандартные постановки задач.

Tang et al. [12] предложили `sched_ext`-планировщик SRAND, в котором расчёт отставаний и виртуальных дедлайнов EEVDF заменён собственной упрощённой схемой виртуального времени и многоуровневой очередью. По завершении кванта виртуальное время задачи увеличивается на величину $(slice - p.slice) \cdot 100/p.weight$, где $slice = 20$ мс — длительность кванта, $p.slice$ — его неиспользованный остаток, $p.weight$ — вес задачи. У задач с большим весом виртуальное время растёт медленнее, что соответствует более высокому приоритету. Только что появившиеся и пробуждающиеся задачи инициализируются текущим глобальным виртуальным временем системы, которое монотонно подтягивается вверх по максимуму активных задач. Если у проснувшейся задачи виртуальное время меньше $V(t) - slice$, оно принудительно поднимается до этого порога, что не даёт долго спавшей задаче монополизировать ядро.

Полученное виртуальное время служит ключом распределения по пяти BPF-очередям с порогами 20, 50, 200 и 500 мс (`queue0–queue4`). Очереди с меньшим индексом опустошаются первыми и попадают в единую глобальную FIFO-очередь, откуда ядра напрямую извлекают следующую задачу, что заменяет обход красно-чёрного дерева EEVDF одной операцией взятия с головы. Дополнительно поддерживаются локальные очереди свободных ядер процессора с привязкой задач к ядрам, на которых они уже работали, а потоки ядра получают неограниченный квант и направляются непосредственно в локальную очередь. Политика реализована через обработчики `select_cpu`, `enqueue` и `dispatch`, время ожидания задачи в очереди ограничено порогом 5000 мс. По данным авторов, на `Lmbench` время переключения контекста сокращается на 11,83%, а на `hackbench` общая производительность растёт на 7,02% при сопоставимой с EEVDF равномерности распределения нагрузки (вариативность времени работы ядер 0,005 против 0,006 у EEVDF).

Xu et al. [13] применили `sched_ext` к задаче управляемого тестирования параллелизма (*controlled concurrency testing*, CCT) ядра Linux. Планировщик LACE сериализует конфликтующие потоки и переключает их в контролируемом порядке, автоматически вставляя точки переключения в операциях с разделяемой памятью и примитивах синхронизации, что позволяет систематически воспроизводить состояния гонки без вмешательства гипервизора. По

сравнению с фаззером SegFuzz LACE достигает на 38% большего покрытия ветвлений, снижает накладные расходы на 57% и ускоряет воспроизведение ошибок в 11,4 раза, обнаружив восемь ранее неизвестных ошибок параллелизма при объёме патчей ядра менее 25 строк. Работа показывает, что `sched_ext` пригоден не только для оптимизации производительности, но и как инструмент верификации корректности параллельного кода.

Mao et al. [14] рассмотрели задачу планирования с точки зрения обеспечения полноты данных о происхождении (*provenance*). При высоком темпе генерации событий EEVDF, LAVD и Rusty теряют от 39% до более чем 98% событий, что компрометирует гарантии эталонного монитора и открывает возможность для угрозы суперпродюсера. Планировщик Aegis строится на двухслойной архитектуре. Первый слой делит задачи на основную и несколько непервичных очередей: для непервичных задаётся время ожидания, играющее роль бюджета ресурсов, а выбор очереди происходит по наиболее *голодной* из них, что обеспечивает справедливость без явных приоритетов. Второй слой управляет переключением через DeepQ-Network с двойным вознаграждением — r_c штрафует за потерю событий и обеспечивает полноту, r_p поощряет утилизацию процессора. На бенчмарках с системами Sysdig и eAudit Aegis показал нулевую потерю событий при суммарных накладных расходах менее 0,5% в типичных режимах и около 2,44% в наиболее тяжёлых, что достигается дельта-функцией пропуска несущественных решений планирования.

Wang et al. [15] развили идею адаптивного планирования до уровня портфеля политик. Предложенный агент Adaptive Scheduling Agent (ASA) декомпозирует задачу на распознавание шаблона нагрузки и сопоставление ему планировщика из набора предобученных экспертов. Шаблон определяется ансамблевым классификатором на основе XGBoost по метрикам утилизации процессора, ввода-вывода и сетевой активности, собираемым через eBPF и `procfs`, а для подавления шумов применяется взвешенное по времени вероятностное голосование с экспоненциальным затуханием. Подготовка агента ведётся в три этапа: прототипное обучение базовой модели, динамическая калибровка стоимости переключения и обучение модели обобщения. Динамическое переключение `sched_ext`-политики выполняется на лету. ASA превосходит EEVDF в 86,4% тестовых сценариев и попадает в тройку лучших политик в 78,9% случаев, а на смешанных нагрузках обгоняет даже статический оракул за счёт переключения политик внутри одной задачи. Агент потребляет около 1,45% одного ядра и 297 МБ памяти и сохраняет качество при переносе на новые аппаратные платформы, не входившие в обучающую выборку. Работа подчёркивает ограниченность статичной политики при росте разнообразия нагрузок и оборудования.

Galín и Hedblom [16] оценили планировщик LAVD (Latency-criticality Aware Virtual Deadline) в telco-окружении Kubernetes Minikube. Поводом к работе послужила проблема задержек в облачной инфраструктуре Ericsson, резко возрастающих при загрузке процессора выше 50%, что делает оценку поведения планировщика на интервале 50–95% критически важной. LAVD изначально проектировался для игровых нагрузок и предлагается для telco в силу схожих требований к задержке. Для воспроизводимой оценки авторы реализовали многопоточный симулятор на C++, генерирующий нагрузки на основе трассировочных данных Ericsson: распределение времени обслуживания подобрано как бета-распределение, давшее наибольшее значение $p = 0,084$ по критерию Колмогорова–Смирнова, плюс отдельный генератор фоновой интерферирующей нагрузки.

В исследовании сопоставлены три планировщика: LAVD, минималистичный `scx_simple` без поддержки вытеснения и штатный EEVDF в роли базовой линии. При высокой доле фоновой интерферирующей нагрузки LAVD сокращает среднее время оборота относительно EEVDF на 6–81% и улучшает 99-й процентиль до 90%. Однако в типичных для Ericsson условиях (50% загрузки трафиком и 10% интерферирующей нагрузки) `scx_simple` стабильно обгоняет LAVD, несмотря на значительно меньшую сложность, а попытки тонкой настройки минимального и максимального кванта LAVD не дают значимых улучшений. Авторы делают вывод, что преимущество принадлежит не отдельной политике, а самой платформе `sched_ext` как средству быстрого сравнения алгоритмов и адаптации планирования к доменной специфике.

Перечисленные работы охватывают оптимизацию очередей, тестирование параллелизма, обучаемые политики и доменную адаптацию, но исходят из однородного вычислительного ресурса. Задача планирования на гетерогенных процессорах в рамках `sched_ext` остаётся значительно менее изученной. Ближе всего к ней стоит работа Van Craeynest et al. [17], в которой интенсивность обращений к памяти признана недостаточным критерием сопоставления задачи типу ядра, а оценка производительности выводится из совместного учёта ILP и MLP на уровне CPI-компонент. Предложенный механизм PIE прогнозирует поведение задачи на альтернативном ядре по CPI-стеку без её миграции и даёт прирост пропускной способности на 5,5% над современными планировщиками и на 8,7% над политиками на основе сэмплирования. Однако работа относится к 2012 году и предлагает аппаратный механизм оценки, тогда как в настоящей работе тот же учёт ведётся программно в рамках `sched_ext`. Заполнение этого пробела — учёт вычислительной мощности ядер в программной политике планирования на современных гетерогенных платформах и составляет предмет настоящей работы.

1.5 Анализ подходов к реализации планировщика

Способ реализации планировщика определяет темп исследования, цену ошибки в политике и переносимость результатов между версиями ядра. К настоящему моменту в практике закрепились четыре подхода: прямая модификация штатного fair-класса, загружаемый модуль ядра, пользовательский агент по схеме ghOSt и eBPF-программа в рамках sched_ext.

Прямая модификация штатного fair-класса обеспечивает наилучшее быстроедействие, поскольку политика выполняется без промежуточных слоёв и имеет полный доступ к внутренним структурам ядра. Однако цикл разработки оказывается дорогим. Каждое изменение требует пересборки ядра (порядка 10 минут на современной машине) и перезагрузки целевой системы, ошибка в логике планирования с высокой вероятностью приводит к панике ядра, зависанию или повреждению состояния системы, а сравнение нескольких вариантов превращается в линейное накопление времени сборки и перезагрузки. Подход оправдан при подготовке итоговой реализации к включению в основную ветку Linux, но плохо подходит для итеративного исследования.

Загружаемый модуль ядра сокращает цикл итерации до секунд, поскольку вместо пересборки ядра достаточно перезагрузки модуля. Однако последствия ошибок остаются прежними, модуль исполняется с привилегиями ядра, и некорректная политика так же способна привести к панике, зависанию или повреждению состояния системы, как и встроенная реализация. Кроме того, в Linux отсутствует официальный интерфейс для замены планировщика fair-класса, поэтому модуль вынужден опираться на внутренние структуры ядра, такие как `struct sched_class` и поля `task_struct`, регулярно меняющиеся между версиями. ABI ядра в общем случае не стабилизирован, и решение оказывается жёстко привязано к конкретной версии ядра и требует сопровождения при каждом обновлении.

Пользовательский агент по схеме ghOSt [11] переносит логику планирования в обычный процесс, что обеспечивает устойчивость к ошибкам политики и широкий выбор языков реализации. Взаимодействие с ядром построено на разделяемой памяти и транзакционной передаче решений, поэтому накладные расходы заметно ниже наивной модели на системных вызовах, однако каждое решение всё равно требует синхронизации между ядром и агентом и проигрывает реализации внутри ядра на нагрузках с короткими и частыми пробуждениями. Помимо этого, ghOSt поставляется отдельным набором патчей ядра, не входящим в основную ветку, поэтому требует сопровождения при каждом обновлении ядра.

eBPF-программа в рамках sched_ext снимает основные ограничения предыдущих подходов. Программа выполняется внутри ядра, обеспечивая сопоста-

вимое со встроенной реализацией быстродействие. Безопасность гарантируется верификатором eBPF на этапе загрузки (см. раздел 1.3), а таймер watchdog при зависании политики автоматически выгружает eBPF-планировщик и возвращает систему под управление EEVDF. Подключение и выгрузка планировщика выполняются на работающей системе за миллисекунды без перезагрузки, а механизм CO-RE обеспечивает переносимость скомпилированной программы между близкими версиями ядра. Существенно и то, что с момента принятия в основную ветку sched_ext стал фактическим стандартом сравнения новых алгоритмов планирования, что упрощает воспроизведение и прямое сопоставление результатов с существующими работами.

Результаты сравнения сведены в таблицу 1.1, где знаки +, – и ± обозначают полное, отсутствующее и частичное выполнение критерия. Устойчивость к ошибкам политики здесь означает отсутствие риска паники, зависания или повреждения состояния ядра при некорректной логике планирования, а поддерживаемый интерфейс замены — наличие официального ABI для подключения внешнего планировщика без правки внутренних структур ядра.

Таблица 1.1: Сравнение подходов к реализации планировщиков

Критерий	В ядре	Модуль	ghOSt	sched_ext
Без пересборки ядра	–	+	±	+
Устойчивость к ошибкам политики	–	–	+	+
Низкие накладные расходы	+	+	–	±
Поддерживаемый интерфейс замены	–	–	±	+
Переносимость между версиями ядра	–	–	±	+

Среди рассмотренных подходов только sched_ext одновременно обеспечивает работу без пересборки ядра, устойчивость к ошибкам политики, поддерживаемый интерфейс замены и переносимость между версиями ядра, проигрывая встроенной реализации лишь по накладным расходам. Сочетание этих свойств делает возможным итеративное исследование на работающей системе и прямое сопоставление результатов с другими работами, поэтому в качестве основы для дальнейшей реализации выбран фреймворк sched_ext.

2 Математические модели планировщиков

2.1 Введение

В настоящей главе формализуются две модели планирования. Первая модель, EEVDF [5], является штатным планировщиком Linux. В настоящей работе она реализована средствами `sched_ext`, что обеспечивает прямое сравнение с предлагаемой моделью на одной платформе исполнения, а её формализация по [5] служит источником нотации для последующих разделов. Вторая модель, аукционная A1349, построена на основе работ по динамическому VCG-механизму распределения ресурсов [18, 19] и адаптирована к задаче процессорного планирования.

Перед описанием моделей приводится сводка обозначений и допущений, общих для настоящей и последующих глав.

2.2 Обозначения и допущения

2.3 Earliest Eligible Virtual Deadline First sloppy

Earliest Eligible Virtual Deadline First (EEVDF) [5] — алгоритм пропорционально-долевого планирования, предложенный Stoica и Abdel-Wahab. Алгоритм является теоретическим основанием класса `SCHED_NORMAL` ядра Linux начиная с версии 6.6 и используется в настоящей работе в качестве базового алгоритма для сравнения.

Системная модель рассматривает один разделяемый ресурс (процессор), время на котором выделяется квантами длительности не более q . Множество клиентов (задач), активных в момент t , обозначается $\mathcal{A}(t)$; каждому клиенту i приписан положительный вес w_i . Идеальная мгновенная доля ресурса для клиента i определяется как

$$f_i(t) = \frac{w_i}{\sum_{j \in \mathcal{A}(t)} w_j}. \quad (2.1)$$

В идеализированной жидкостной модели при $q \rightarrow 0$ обслуживание $S_i(t_0, t_1)$, причитающееся клиенту i на интервале $[t_0, t_1]$, есть интеграл от f_i :

$$S_i(t_0, t_1) = \int_{t_0}^{t_1} f_i(\tau) d\tau. \quad (2.2)$$

Разность между причитающимся и фактически полученным обслуживанием

задаёт центральную метрику справедливости — отставание (lag):

$$\text{lag}_i(t) = S_i(t_0^i, t) - s_i(t_0^i, t), \quad (2.3)$$

где t_0^i — момент входа клиента i в систему, s_i — фактически полученное обслуживание. Положительный lag означает недообслуженного клиента, отрицательный — переобслуженного.

Виртуальное время системы $V(t)$ вводится как

$$V(t) = \int_0^t \frac{1}{\sum_{j \in \mathcal{A}(\tau)} w_j} d\tau. \quad (2.4)$$

Скорость роста V обратно пропорциональна суммарному весу активных клиентов: при увеличении конкуренции виртуальное время замедляется. На одну единицу виртуального времени каждый активный клиент накапливает ровно w_i единиц реального обслуживания. В этих обозначениях обслуживание линейно по виртуальному времени:

$$S_i(t_1, t_2) = w_i \cdot (V(t_2) - V(t_1)). \quad (2.5)$$

Каждый запрос k клиента i характеризуется длиной $r^{(k)}$, виртуальным временем доступности $ve^{(k)}$ и виртуальным дедлайном $vd^{(k)}$. Запрос считается *доступным* (eligible) в момент t , если $ve \leq V(t)$. Время доступности выбирается так, чтобы в жидкостной модели обслуживание, причитающееся клиенту до момента e , совпадало с реально полученным:

$$ve = V(t_0^i) + \frac{s_i(t_0^i, t)}{w_i}. \quad (2.6)$$

Виртуальный дедлайн назначается так, чтобы за интервал $[ve, vd]$ в жидкостной системе клиенту было доставлено ровно r единиц обслуживания:

$$vd = ve + \frac{r}{w_i}. \quad (2.7)$$

Для последовательных запросов выполняется $ve^{(k+1)} = vd^{(k)}$ при полном использовании предыдущего кванта; при частичном использовании $u^{(k)} \leq r^{(k)}$ обновление принимает вид $ve^{(k+1)} = ve^{(k)} + u^{(k)}/w_i$, что предотвращает потерю неиспользованного времени и удерживает lag вблизи нуля.

Правило выбора очередной задачи формулируется в две стадии. Сначала применяется фильтр доступности $ve \leq V(t)$, затем среди доступных запросов выбирается тот, у которого виртуальный дедлайн vd минимален. В практической реализации виртуальное время системы V явно не поддерживается — его роль играет упорядочивание в красно-чёрном дереве по полю vd , дополненном

агрегированной информацией о минимальном vd среди доступных запросов в каждом поддереве, что обеспечивает выбор задачи за $O(\log n)$.

При динамических событиях (вход и выход клиентов, изменение весов) виртуальное время системы корректируется для сохранения закона сохранения отставаний: $\sum_{i \in \mathcal{A}(t)} \text{lag}_i(t) = 0$. При выходе клиента j в момент t

$$V(t^+) = V(t) + \frac{\text{lag}_j(t)}{\sum_{i \in \mathcal{A}(t^+)} w_i}, \quad (2.8)$$

при входе клиента с отставанием $\text{lag}_j(t)$

$$V(t^+) = V(t) - \frac{\text{lag}_j(t)}{\sum_{i \in \mathcal{A}(t^+)} w_i}. \quad (2.9)$$

Если клиент входит и выходит при нулевом отставании, виртуальное время непрерывно.

Для EEVDF доказана теорема об ограничении отставания: для любого активного клиента k в стационарном режиме

$$-r_{\max} < \text{lag}_k(t) < \max(r_{\max}, q), \quad (2.10)$$

где $r_{\max}$ — максимальная длина запроса клиента k . При $r^{(k)} \leq q$ для всех запросов оценка усиливается до $|\text{lag}_k(t)| < q$, и эта граница является точной. Отсюда вытекает оптимальность EEVDF среди пропорционально-долевых алгоритмов с равными квантами в смысле минимизации максимального отставания [5].

2.4 Аукционная модель A1349 sloppy

Базовая модель EEVDF предполагает однородный вычислительный ресурс с фиксированной мощностью каждого ядра. В современных гетерогенных процессорах (Intel P/E, ARM big.LITTLE) разница в производительности между классами ядер достигает двух-трёх раз. Прямая трактовка таких ядер как одинакового ресурса приводит к неоптимальному распределению задач: чувствительные к латентности задачи могут оказаться на медленных ядрах, а фоновые — на быстрых. Поэтому для гетерогенной платформы в настоящей работе предлагается аукционная модель планирования A1349, в которой выбор ядра трактуется как задача распределения недолговечного ресурса между конкурирующими агентами.

Гетерогенная платформа описывается двумя множествами ядер: $\mathcal{P} = \{P_1, \dots, P_{K_P}\}$ — ядра высокой производительности (P-cores) и $\mathcal{E} = \{E_1, \dots, E_{K_E}\}$ — энергоэффективные ядра (E-cores). Время разделено на

дискретные кванты $t = 1, 2, \dots$. В каждый момент времени каждое ядро представляет собой *недолговечный* (perishable) ресурс: неиспользованный квант теряется безвозвратно.

Каждая задача $i \in \mathcal{N}^t$ выступает в роли агента-покупателя со скрытым типом $\theta_i = (v_i, l_i)$, где $v_i \in [0, \bar{V}]$ — ценность завершения задачи для пользователя, $l_i \in \{1, \dots, L\}$ — требуемое число квантов на Р-ядре. Поскольку Р- и Е-ядра разделяют общий набор инструкций, задача может быть назначена на любой класс ядер; выбор типа — решение механизма, а не задачи. На Е-ядре длина задачи увеличивается в $\sigma > 1$ раз и составляет $\lceil l_i \cdot \sigma \rceil$ квантов, где $\sigma = \eta_P / \eta_E$ выражает отношение производительностей. Задача i получает полезность v_i только при исполнении полного контракта длиной m_i квантов.

Каждой задаче приписан бюджет $B_i > 0$, определяемый её классом обслуживания (real-time, interactive, batch, background). Задача может выполняться, пока суммарные платежи $\sum_s p_i^s$ не превышают B_i . Принадлежность к классу κ_i задаётся стандартными механизмами ОС (поле `policy` и `nice`-значение) и не входит в скрываемый тип θ_i .

Процесс планирования формализуется как марковский процесс принятия решений (MDP). Состояние в момент t

$$s^t = (\mathbf{x}^t, \mathbf{b}^t, \boldsymbol{\omega}^t), \quad (2.11)$$

где $\mathbf{x}^t \in \{0, \dots, L-1\}^K$ — остаточные длительности контрактов на каждом из $K = K_P + K_E$ ядер, $\mathbf{b}^t$ — остаточные бюджеты активных задач, $\boldsymbol{\omega}^t$ — внешние факторы (температура, потребление, занятость памяти). Действие a^t задаёт для каждой задачи распределение по типам ядер $a_i^t = (a_{i,P}^t, a_{i,E}^t) \in \{0, 1\}^2$, $a_{i,P}^t + a_{i,E}^t \leq 1$, с ограничениями

$$\sum_{i \in \mathcal{N}^t} a_{i,P}^t \leq K_P^t, \quad \sum_{i \in \mathcal{N}^t} a_{i,E}^t \leq K_E^t, \quad (2.12)$$

где K_P^t, K_E^t — число свободных ядер каждого типа.

Базовая стоимость одного кванта на ядре типа $\kappa \in \{P, E\}$ обозначается c_P, c_E , причём $c_P > c_E$; отношение $\gamma = c_P / c_E > 1$ отражает различие в производительности. Полная стоимость контракта для задачи i :

$$\text{cost}(i, P) = c_P \cdot l_i, \quad \text{cost}(i, E) = c_E \cdot \lceil l_i \cdot \sigma \rceil. \quad (2.13)$$

Соотношение нелинейно: при $\lceil l_i \sigma \rceil / l_i < \gamma$ Е-ядро оказывается дешевле, иначе — дороже. Это создаёт нетривиальную задачу выбора типа ядра, не сводящуюся к жадной стратегии.

Эффективная стоимость задачи на каждом типе ядра с учётом стоимости квантов:

$$\phi_P(\theta_i) = v_i - c_P \cdot l_i, \quad \phi_E(\theta_i) = v_i - c_E \cdot [l_i \cdot \sigma]. \quad (2.14)$$

Расширенная социальная функция благосостояния задаётся уравнением Беллмана:

$$W(s^t) = \max_{a^t} \left[\sum_{i \in \mathcal{N}^t} (a_{i,P}^t \phi_P(\theta_i) + a_{i,E}^t \phi_E(\theta_i)) + \delta \cdot \mathbb{E}[W(s^{t+1})] \right], \quad (2.15)$$

где $\delta \in (0, 1)$ — коэффициент дисконтирования. Целевая функция оптимизации включает штраф за несправедливость:

$$\text{SC}(\pi) = W(\pi) - \lambda_F \cdot \Psi(\pi), \quad \Psi(\pi) = \text{Var}_i \left[\frac{u_i(\pi)}{v_i} \right], \quad (2.16)$$

где u_i — полезность задачи i при политике π , $\lambda_F \geq 0$ — параметр баланса между общей эффективностью и равномерностью распределения.

Платёж задачи i , получившей ядро типа κ в момент t , определяется по правилу VCG (Викри–Кларка–Гровса) как разность между социальным благом без её участия и вкладом в текущее распределение:

$$p_i^t = W_{-i}(s^t) - W(s^t) + \phi_\kappa(\theta_i). \quad (2.17)$$

Интуитивно задача оплачивает экстерналию, налагаемую ею на остальные активные задачи. В частном случае одного свободного ядра платёж сводится к

$$p_{i^*} = \phi_\kappa(\theta_j) + (\delta^{m_\kappa(l_j)} - \delta^{m_\kappa(l_{i^*})}) \bar{W}_\kappa, \quad (2.18)$$

где j — следующая по эффективной стоимости задача, $\bar{W}_\kappa$ — ожидаемое будущее благосостояние в зависимости от типа ядра.

При неизвестных распределениях типа задач и переходного ядра MDP механизм обучается онлайн: горизонт времени разбивается на эпизоды возрастающей длительности, в каждом из которых последовательно проводятся фаза перемешивания (без сбора платежей) и стационарная фаза (с применением текущей политики и сбором оценок). Оценки $\hat{R}, \hat{P}$ строятся по принципу верхней доверительной границы и обновляют политику между эпизодами. Для алгоритма доказаны субквадратичные оценки сожаления (regret) по социальному благосостоянию и платежам порядка $\tilde{O}(T^{2/3})$.

Ключевые свойства полученного механизма:

- *совместимость стимулов в классе* — при фиксированном классе обслуживания κ_i правдивое сообщение типа (v_i, l_i) является доминирующей

стратегией для агента; межклассовая совместимость обеспечивается тем, что класс задаётся ядром ОС вне канала сообщений механизма;

- *индивидуальная рациональность* — полезность задачи при участии неотрицательна, $u_i(\pi^*, p^*) \geq 0$;
- *бюджетная допустимость* — если VCG-платёж превышает остаточный бюджет B_i^t , задача не получает ядро в данном кванте; ограничение реализуется изменением правила распределения, а не платежа, что сохраняет совместимость стимулов внутри класса.

Таким образом, модель A1349 расширяет идею пропорционально- долевого планирования аукционным механизмом, учитывающим гетерогенность ядер, бюджетные ограничения задач и динамику системы во времени.

3 Реализация алгоритмов планирования

4 Описание тестового окружения и экспериментального оборудования

5 Анализ результатов эксперимента

Заключение

Список литературы

- [1] John Lions A Commentary on the Sixth Edition UNIX Operating System The University of New South Wales 1977
- [2] Mike Kravetz, Hubertus Franke, Shailabh Nagar, Rajan Ravindran Enhancing Linux Scheduler Scalability 2001
- [3] Robert Love Linux Kernel Development 2nd ed Novell Press, 2005
- [4] C. S. Wong, I. K. T. Tan, R. D. Kumari, J. W. Lam, W. Fun Fairness and Interactive Performance of O(1) and CFS Linux Kernel Schedulers International Symposium on Information Technology 2008
- [5] Ion Stoica, Hussein Abdel-Wahab Earliest Eligible Virtual Deadline First : A Flexible and Accurate Mechanism for Proportional Share Resource Allocation Old Dominion University 1996
- [6] Greenhalgh P. Big.LITTLE Processing with ARM Cortex-A15 & Cortex-A7 ARM White Paper, 2011
- [7] Rotem E. et al. Alder Lake Architecture 2021 IEEE Hot Chips 33 Symposium (HCS), Palo Alto, CA, USA, 2021, pp. 1–23, doi: 10.1109/HCS52781.2021.9567097
- [8] Conti F., Paulin G., Garofalo A., Rossi D., Pullini A., Benini L. Marsellus: A Heterogeneous RISC-V AI-IoT End-Node SoC with 2-to-8b DNN Acceleration and 30%-Boost Adaptive Body Biasing IEEE Journal of Solid-State Circuits, 2024
- [9] Energy Aware Scheduling [Электронный ресурс]. Linux Kernel Documentation. Режим доступа: <https://www.kernel.org/doc/html/latest/scheduler/sched-energy.html> – Дата обращения 14.11.2025
- [10] Corbet J. The BPF extensible scheduler class LWN.net, 30 November 2022 Режим доступа: <https://lwn.net/Articles/916291/> – Дата обращения 20.11.2025
- [11] Humphries J. T., Natu N., Chaugule A., Weisse O., Rhoden B., Don J., Rizzo L., Rombakh O., Turner P., Kozyrakis C ghOST: Fast & Flexible User-Space Delegation of Linux Scheduling SOSP 2021
- [12] Tang Q., Gao X., Li G., Lin J Optimizing Linux Scheduling Based on Global Runqueue with SCX IEEE International Conference SMC 2024
- [13] Xu J., Wolff D., Yi H. X., Li J., Roychoudhury A. Concurrency Testing in the Linux Kernel via eBPF arXiv 2025

- [14] Mao J., Ujcich B. E., Ma S Rethinking Provenance Completeness with a Learning-Based Linux Scheduler arXiv 2025
- [15] Wang X., Jia S., Huang Z., Cao J., Song M Mixture-of-Schedulers: An Adaptive Scheduling Agent as a Learned Router for Expert Policies arXiv 2025
- [16] Galin M., Hedblom A Linux Kernel Scheduler Evaluation for Performance-Critical Telecom Workloads Linköping University 2025
- [17] Van Craeynest K., Jaleel A., Eeckhout L., Narvaez P., Emer J. S Scheduling Heterogeneous Multi-Cores through Performance Impact Estimation (PIE) ISCA, ACM, 2012, pp. 213–224
- [18] Sano R. A Dynamic Mechanism Design for Scheduling with Different Use Lengths Institute of Economic Research, Kyoto University, 2013
- [19] Leon V., Etesami S. R. Online Learning for Dynamic Vickrey-Clarke-Groves Mechanism in Unknown Environments arXiv:2506.19038, 2025
- [20] Joseph Y-T. Leung Handbook of Scheduling: Algorithms, Models, and Performance Analysis CRC Press 2004
- [21] Michelle Galin, Anna Hedblom Linux Kernel Scheduler Evaluation for Performance-Critical Telecom Workloads Linköping University 2025
- [22] Jinsong Mao, Benjamin E. Ujcich, Shiqing Ma Rethinking Provenance Completeness with a Learning-Based Linux Scheduler arXiv 2025
- [23] Xinbo Wang, Shian Jia, Ziyang Huang, Jing Cao, Mingli Song Mixture-of-Schedulers: An Adaptive Scheduling Agent as a Learned Router for Expert Policies arXiv 2025
- [24] A1349 Pavel Shago [Электронный ресурс]. – Режим доступа: <https://github.com/shgpavel/A1349> – Дата обращения 16.12.2025.
- [25] Andrew S. Tanenbaum, Herbert Bos MODERN OPERATING SYSTEMS Pearson Education 2023
- [26] Liz Rice Learning eBPF O'Reilly Media, Inc 2023
- [27] Torvalds L. Linux Kernel [Электронный ресурс] / L. Torvalds. – Режим доступа: <https://github.com/torvalds/linux> – Дата обращения 04.12.2025.
- [28] sched-ext/scx [Электронный ресурс]. – Режим доступа: <https://github.com/sched-ext/scx> – Дата обращения 07.12.2025.
- [29] lkml [Электронный ресурс]. – Режим доступа: <https://lkml.org> – Дата обращения 24.11.2025.